



PDF Download  
3706628.3708864.pdf  
07 April 2026  
Total Citations: 9  
Total Downloads: 3187



Published: 27 February 2025

Citation in BibTeX format

FPGA '25: The 2025 ACM/SIGDA  
International Symposium on Field  
Programmable Gate Arrays  
February 27 - March 1, 2025  
CA, Monterey, USA

Conference Sponsors:  
SIGDA

DL Latest updates: <https://dl.acm.org/doi/10.1145/3706628.3708864>

RESEARCH-ARTICLE

## FlightVGM: Efficient Video Generation Model Inference with Online Sparsification and Hybrid Precision on FPGAs

JUN LIU, Shanghai Jiao Tong University, Shanghai, China

SHULIN ZENG, Tsinghua University, Beijing, China

LI DING, Shanghai Jiao Tong University, Shanghai, China

WIDYADEWI SOEDARMADJI, Tsinghua University, Beijing, China

HAO ZHOU, Shanghai Jiao Tong University, Shanghai, China

ZEHAO WANG, Tsinghua University, Beijing, China

View all

Open Access Support provided by:

Shanghai Jiao Tong University

Tsinghua University

Infinigence AI



# FlightVGM: Efficient Video Generation Model Inference with Online Sparsification and Hybrid Precision on FPGAs

Jun Liu<sup>\*†</sup>  
Shanghai Jiao Tong University  
Shanghai, China

Shulin Zeng<sup>\*†</sup>  
Tsinghua University  
Beijing, China

Li Ding  
Shanghai Jiao Tong University  
Shanghai, China

Widyadewi Soedarmadji  
Tsinghua University  
Beijing, China

Hao Zhou  
Shanghai Jiao Tong University  
Shanghai, China

Zehao Wang  
Tsinghua University  
Beijing, China

Jinhao Li  
Shanghai Jiao Tong University  
Shanghai, China

Jintao Li  
Yadong Dai  
Kairui Wen  
Infinigence-AI  
Shanghai, China

Shan He  
Yaqi Sun  
Infinigence-AI  
Shanghai, China

Yu Wang<sup>‡</sup>  
Tsinghua University  
Beijing, China

Guohao Dai<sup>‡†</sup>  
Shanghai Jiao Tong University  
Shanghai, China

## Abstract

Video Generation Model (VGM), as a representative of multi-modal large models, has revolutionized the productivity of video content creation. VGMs are compute-bound due to adopting the Diffusion Transformer (*i.e.*, DiT) structure. Sparsification is a common method for accelerating compute-intensive models. Still, sparse VGMs cannot fully exploit the effective throughput (*i.e.*, TOPS<sup>1</sup>) of GPUs. FPGAs are good candidates for accelerating sparse deep learning models. However, existing FPGA accelerators still face low throughput (<2TOPS) on VGMs due to the significant gap in peak computing performance (PCP) with GPUs (>21×). To achieve a higher throughput than GPUs, FPGA-based acceleration of sparse VGMs still faces the following challenges: large redundancy in activations, low performance of DSPs under hybrid precision, and under-utilization using static compilation for online compression.

To tackle these challenges, we propose **FlightVGM**, the first FPGA accelerator for efficient VGM inference with activation sparsification and hybrid precision. In FlightVGM, our motivation stems from VGMs exhibiting different compression preferences in various dimensions and layers. To exploit the video frames' similarity in

the temporal and spatial dimensions, we propose a *spatial-temporal* online activation sparsification architecture, reducing the computational cost by 3.17×. To provide a good trade-off between the accuracy and efficiency of VGMs, we employ fixed-point precision for linear layers and retain floating-point precision for attention layers. Then, we propose a *floating-fixed* hybrid precision DSP58 expansion architecture on the AMD V80 FPGA, boosting the PCP by 3.26×. Finally, to make FlightVGM available to various workloads, we propose a *dynamic-static* combined adaptive scheduling method for low-overhead online sparsification, improving the computation utilization by 2.75×. Implemented on the AMD V80 FPGA, FlightVGM surpasses NVIDIA 3090 GPU by 1.30× in performance and 4.49× in energy efficiency on various sparse VGM workloads.

## CCS Concepts

• **Hardware** → **Hardware accelerators**; • **Computer systems organization** → **Reconfigurable computing**.

## Keywords

Video Generation Model, FPGA, Acceleration

## ACM Reference Format:

Jun Liu, Shulin Zeng, Li Ding, Widyadewi Soedarmadji, Hao Zhou, Zehao Wang, Jinhao Li, Jintao Li, Yadong Dai, Kairui Wen, Shan He, Yaqi Sun, Yu Wang, and Guohao Dai. 2025. FlightVGM: Efficient Video Generation Model Inference with Online Sparsification and Hybrid Precision on FPGAs. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '25)*, February 27–March 1, 2025, Monterey, CA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3706628.3708864>

## 1 Introduction

Recently, Video Generation Models (VGMs) [42] have attracted extensive attention in the field of Artificial Intelligence Generated

<sup>\*</sup>Both authors contributed equally to this research.

<sup>†</sup>Also with Infinigence-AI.

<sup>‡</sup>Corresponding authors. Email: yu-wang@tsinghua.edu.cn, daiguohao@sjtu.edu.cn

<sup>1</sup>We represent FP16 and INT8 throughput and computing performance in TOPS.

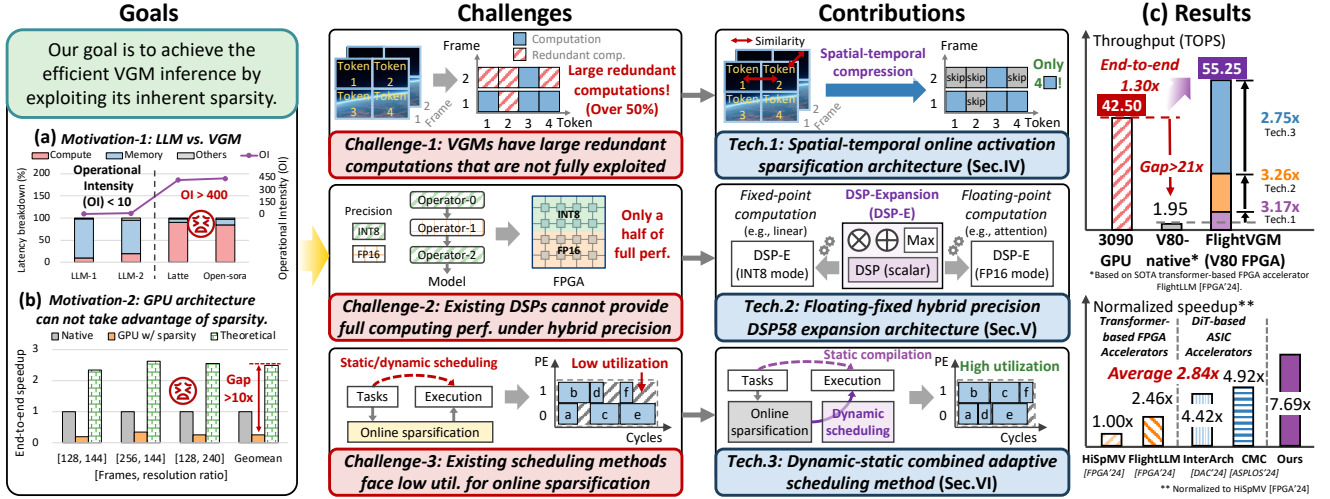
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

FPGA '25, February 27–March 1, 2025, Monterey, CA, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1396-5/25/02

<https://doi.org/10.1145/3706628.3708864>



**Figure 1: Overview of the challenges and contributions. The motivations are: (a) VGMs are compute-bound due to the DiT structure, and (b) existing GPUs are very inefficient on sparse VGMs. For the (c) results, FlightVGM surpasses NVIDIA 3090 GPU by 1.30 $\times$  in performance through a series of customized designs.**

Content (AIGC) [20]. Following the emergence of Large Language Models (LLMs) [11], VGMs are regarded as the second milestone on the road towards Artificial General Intelligence (AGI) [27]. VGMs can provide users with creative videos that meet their requirements under appropriate prompts. The Sora model [16], released by OpenAI in February 2024, is the first VGM model that generates a minute of high-fidelity video. Technology companies, such as Google, ByteDance, and Kuaishou, have respectively launched a series of VGMs, including Veo [18], Dreamina [12], Kling [13], and so on. Mainstream VGMs are developed based on the Diffusion Transformer (*i.e.*, DiT) structure [46], which follows the scaling law and shows more capabilities as more data is available [43, 64, 65, 72].

Unlike the memory-intensive nature of transformer-based LLMs [11], DiT-based VGMs are compute-intensive. This is because the DiT structure introduces several compute-bound operations. For example, the linear layers in DiT, including the QKVO projection and Feed Forward Network (FFN), have an additional batch dimension. As shown in Fig. 2, a DiT block consists of a temporal transformer and a spatial transformer. The former explores the relationship between different frames of a video, while the latter explores the relationship between different patches on the same frame. For the spatial/temporal transformer, the temporal/spatial dimension is viewed as the batch, with sizes on the order of tens to hundreds. Fig. 1(a) shows that the Operational Intensity (*i.e.*, FLOPs/Byte) [61] of VGMs is more than 40 $\times$  that of LLMs.

Sparsification is widely used to accelerate compute-intensive deep learning models [29, 30, 35, 60, 63], but sparse VGMs struggle to fully exploit the performance (*i.e.*, TOPS) of GPUs. VGMs exhibit significant redundancy in activations due to the intrinsic similarity of the generated videos in the temporal and spatial dimensions. GPUs adopt the Single-Instruction-Multiple-Thread (SIMT) architecture and are therefore suitable for highly parallelized scenarios. However, the sparsity in activations introduces irregular memory access and synchronization processing, leading to non-negligible overhead on GPUs. Fig. 1(b) shows that there is a huge gap of up

to 10 $\times$  between the actual and theoretical speedup of the NVIDIA 3090 GPU, under different video sizes of VGMs.

FPGAs are good candidates for accelerating sparse deep learning models [29, 38, 41, 47, 52, 66]. However, due to the significant difference in peak computing performance (PCP), there is still a large gap between the throughput of FPGAs and GPUs. Taking the latest AMD V80 FPGA [9] as an example, it offers only 6.5TOPS FP16 PCP, while the NVIDIA 3090 GPU provides 142TOPS FP16 PCP. We evaluate the state-of-the-art (SOTA) transformer-based FPGA accelerator (*i.e.*, FlightLLM [66]) on the V80 FPGA, whose throughput is <2TOPS with 6.5TOPS FP16 PCP. Fig. 1(c) shows that there is a large gap (>21 $\times$ ) between the throughput of FlightLLM and that of the NVIDIA 3090 GPU (~43TOPS). To fully unleash the potential of FPGA acceleration for sparse VGMs, there are still several challenges to be addressed:

**Challenge-1: VGMs have a large number of redundant computations that are not fully exploited.** The similarity of activations exists in both temporal and spatial dimensions simultaneously. However, existing SOTA DiT-based accelerators [53, 59] only exploit a part of the similarity (*i.e.*, temporal or spatial) in VGMs, resulting in less than 46.44% sparsity. Fig. 1(b) shows that, when running at such a low sparsity, the throughput of sparse VGMs on the GPU is even worse than that of dense VGMs.

**Challenge-2: DSP-based processing engines cannot provide full computing performance under hybrid precision.** Different layers of the VGM have different sensitivities to computational precision [40, 70]. For example, VGMs favor INT8 for linear operations and FP16 for attention map operations. However, the DSP58 on AMD V80 FPGA can only be configured in either the FP16 or the INT8 mode [6], leading to a fixed-floating separated processing engine (PE) design with severe under-utilization.

**Challenge-3: Existing scheduling methods face low computational utilization for online sparsification.** The existing scheduling methods include static scheduling [31, 44, 68, 69] and dynamic scheduling [22, 45, 57]. Static scheduling methods rely on

offline operator latency prediction, which is unavailable in online sparsification. Dynamic scheduling methods incur unacceptable additional overhead due to the limited performance of the embedded CPU on FPGAs and complex sparse DiT structures. Since the redundancy of VGMs is present in continuously changing video, existing scheduling methods cannot efficiently handle sparse activations that change dynamically at runtime, leading to low computational utilization for sparse VGMs.

To address these challenges, we propose **FlightVGM**, an FPGA-based accelerator with online sparsification and hybrid precision for efficient VGM inference without degradation in video quality. The main contributions of this paper are summarized as follows:

- To the best of our knowledge, this paper is the first FPGA accelerator to accelerate video generation models, with both performance and energy efficiency exceeding that of GPUs.
- We propose a spatial-temporal online activation sparsification architecture for *Challenge-1*. FlightVGM introduces customized sparsification and recovery units to dynamically prune activations using spatial-temporal similarity, which reduces overhead by  $3.17\times$  against original VGMs.
- We propose a floating-fixed hybrid precision DSP58 expansion architecture for *Challenge-2*. Implemented on the AMD V80 FPGA for INT8/FP16 quantized VGMs, FlightVGM boosts  $3.26\times$  PCP improvement over FP16-only DSP-native design.
- We propose a dynamic-static combined adaptive scheduling method for *Challenge-3*. By introducing the weight preloading and priority-based scheduling mechanism for sparse activation, FlightVGM improves the overall utilization by  $2.75\times$  compared with existing static and dynamic scheduling methods, with negligible scheduling overhead.

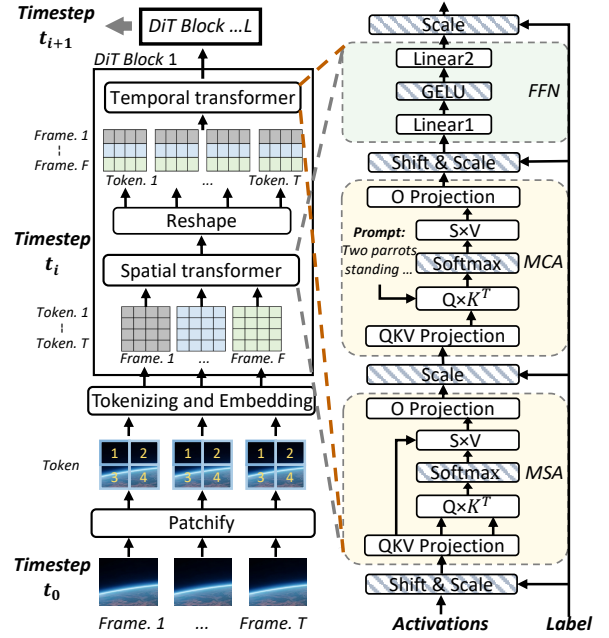
Our results on multiple VGMs (Latte-1 [43] and Open-Sora 1.2 [72]) show that, compared with the NVIDIA 3090 GPU, FlightVGM achieves an improvement of  $1.30\times$  performance and  $4.49\times$  energy efficiency. FlightVGM further achieves average  $2.84\times$  performance improvement of compared with SOTA customized accelerators.

## 2 Backgrounds and Observations

### 2.1 Preliminaries

Diffusion models [32] can restore noisy frames into clear frames described by specific prompts, by gradually predicting Gaussian noise and denoising [50] within multiple timesteps. Current advanced VGMs mostly use the Diffusion Transformer (DiT) [46] as the backbone network to achieve noise prediction. This is due to the strong scalability of the DiT structure for generating long-duration and high-resolution videos. VGMs mainly consist of many cascaded DiT blocks, each including a spatial transformer and a temporal transformer. Both spatial and temporal transformers have three main types of networks: Multi-Head Self-Attention (MSA), Multi-Head Cross-Attention (MCA), and Feed Forward Network (FFN).

As shown in Fig. 2, we take the Open-Sora 1.2 model [72] as an example, which is a typical DiT-based VGM. At a certain timestep  $t_i$ , for a noisy video with 2 channels  $z_i \in \mathbb{R}^{2 \times F \times H \times W}$  consisting of  $F$  frames of size  $H \times W$ . The number of channels represents the two dimensions of the noise to be predicted. Here, we mainly show the process of noise prediction with one dimension.  $z_i$  is first decomposed into multiple patches with position information. The



**Figure 2: DiT block is the main component of VGMs. Here, we show the noise prediction in a DiT block of timestep  $t_i$ .**

size of each patch is  $P \times P$ , and the number of patches for each frame  $T = \frac{H \times W}{P^2}$ . Then, each of these patches is tokenized into the vector (token), and these tokens form the input activation  $Act_i^{F \times T \times d}$  with  $FT$  tokens of size  $1 \times d$  ( $d$  is the hidden dimension).

In each DiT block, the activation  $Act_i^{F \times T \times d}$  is the first input into the spatial transformer for intra-frame computing. The MSA projects  $Act_i^{F \times T \times d}$  as multi-head query  $Q$ , key  $K$ , and value  $V$  and performs attention computation as illustrated in Fig. 2. The MCA is similar to MSA, which embeds the influence of prompts into  $Act_i$  through  $K$  and  $V$  projection. After MCA, the FFN further transforms  $Act_i$  through two fully connected layers with a GELU function. The *Label* information (e.g., timestep, fps, etc.) is embedded into the activation through shift and scale operations. The process of the temporal transformer is the same as that of the spatial transformer. Through  $L$  DiT blocks, VGM predicts the noise of  $t_i$  and obtains the noisy video of  $t_{i+1}$  by denoising. After multiple timesteps of denoising, VGM generates the final video guided by the prompts.

### 2.2 Compression Method on DiT-based VGMs

**2.2.1 Activation Sparsification.** Recent studies [53, 59] focus on video understanding, without fully exploring the similarity in video generation. InterArch [59] proposes a coarse-grained inter-frame sparsification method to reduce redundant information. CMC [53] proposes a method using the GPU video processing unit CODEC [55] to predict inter-frame redundant information. However, both InterArch and CMC mainly compress the computations from a single temporal dimension and thus have limited acceleration for VGMs.

**2.2.2 Hybrid Precision Quantization.** Several studies [40, 70, 71] perform hybrid precision quantization on DiT-based VGMs due to the favorable balance between model accuracy and cost savings.



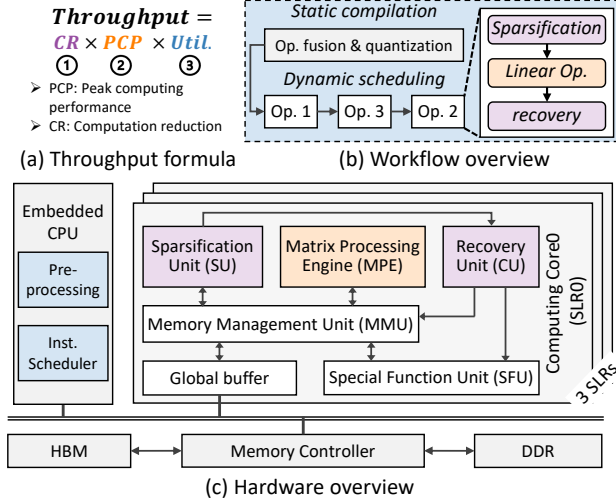


Figure 3: The overview of FlightVGM.

For example, the ViDiT-Q method [70] achieves INT8 precision within DiT blocks, while maintaining attention map computations and other modules outside DiT at FP16 precision. ViDiT-Q employs a uniform symmetric approach for INT8 quantization. However, current hybrid precision quantization methods do not perform effectively on FPGAs. This limitation arises because FPGA DSPs can only operate in FP16 or INT8 precision during runtime. As a result, this constraint leads to reduced PCP when hybrid precision quantization methods are deployed directly on FPGAs. For instance, the INT8 PCP of the V80 FPGA, which supports hybrid precision, is only 66.67% of its maximum hardware PCP (as 1/3 of the DSPs are configured as floating-point units).

### 3 Design Methodology Overview

#### 3.1 FlightVGM Overview

The goal of FlightVGM is to achieve the high throughput of efficient VGM inference on FPGAs. We aim to optimize *CR* (computation reduction), *PCP*, and *Util.* (utilization) through a full-stack solution guided by the throughput formula shown in Fig. 3(a).

(1) **Reducing computation cost:** FlightVGM dynamically prunes activations using the spatial-temporal similarity (detailed in Sec.4), with the customized sparsification unit and recovery unit (highlighted in purple) to ensure low-overhead online sparsification.

(2) **Boosting PCP:** FlightVGM implements a floating-fixed hybrid precision DSP58 expansion architecture (detailed in Sec.5) in the matrix processing engine (highlighted in orange), providing sufficient INT8 and FP16 performance in VGMs.

(3) **Improving computing utilization:** FlightVGM employs a two-stage sparsity-aware scheduling (detailed in Sec.6 and highlighted in blue). Static compilation implements the operator fusion and weight quantization. Dynamic scheduling adjusts the execution order between operators to maximize utilization with sparsification.

#### 3.2 Hardware Architecture Overview

As shown in Fig. 3(c), the overall hardware architecture of FlightVGM consists of three parts: the embedded CPU, off-chip memory, and on-chip execution part. The on-chip execution part includes three

computing cores. Each computing core corresponds to one Super Logic Region (SLR) and includes a Sparsification Unit (SU), a Matrix Processing Engine (MPE), a Recovery Unit (RU), and a Special Function Unit (SFU). The SU exploits the similarity between and within frames to achieve activation sparsification and computation reduction. The MPE comprises a heterogeneous DSP array supporting FP16-INT8 hybrid precision to efficiently execute the compressed matrix multiplications. The RU recovers the output activation according to the index tables generated by the SU. The SFU supports non-linear operations (e.g., softmax, layernorm, and GELU). The Memory Management Unit (MMU) handles data transmission between the global buffer and the computing units. The embedded CPU (e.g., the dual-core A72 processor on the V80 FPGA [9]) is in charge of online instruction scheduling for workload balancing. The off-chip memory part mainly consists of a memory controller, high-bandwidth memory (HBM), and DDR.

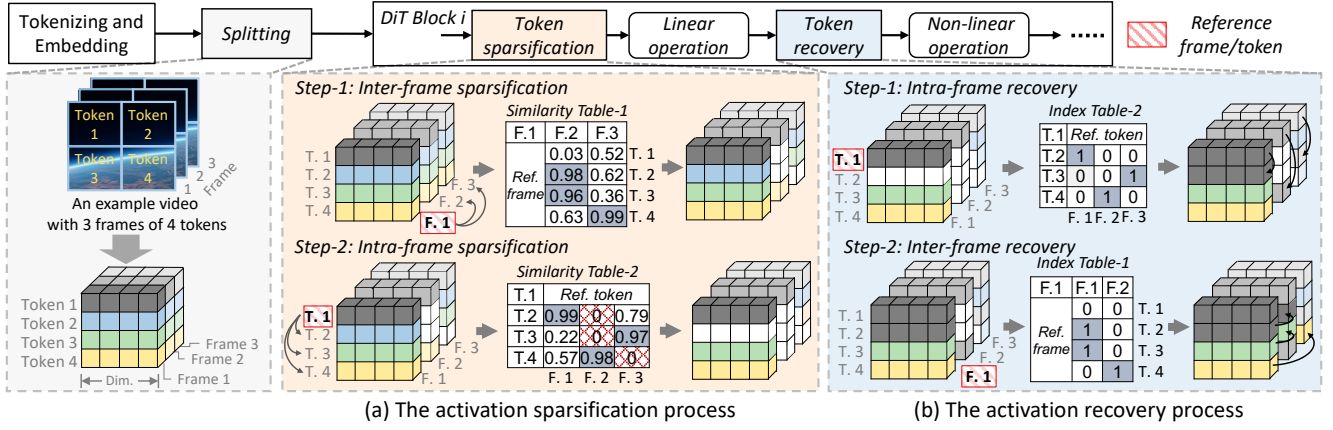
## 4 Spatial-Temporal Activation Sparsification

### 4.1 Sparsification Method

We propose the spatial-temporal activation sparsification method that simultaneously considers inter-frame and intra-frame similarity. The essence of activation sparsification is that *if there is a high degree of similarity between two tokens, then we can calculate only one of the tokens and share the result with the other token*. As shown in Fig. 4, the input activation is a 3-dimensional tensor processed by the tokenizer from a noisy video. Therefore, the input activation can be expressed as  $Act_{in} \in \mathbb{R}^{F \times T \times d}$ , where  $F$  is the number of frames,  $T$  is the number of tokens, and  $d$  is the hidden dimension. For ease of description, we use  $(F_1, T_1)$  to represent the 1st token on the 1st frame. For the input activation  $Act_{in}$ , we use  $V_{ref} \in \mathbb{R}^d$  and  $V_{in} \in \mathbb{R}^d$  to represent the reference vector and the input vector. The formula for similarity calculation is as follows:

$$Sim = \frac{V_{ref} \cdot V_{in}}{\|V_{ref}\| \cdot \|V_{in}\|} \quad (1)$$

**4.1.1 Activation sparsification.** Activation sparsification consists of two steps, namely inter-frame sparsification and intra-frame sparsification. For *inter-frame sparsification*, we first separate the input activation into groups of consecutive  $G$  frames. For each group, we select one frame as the reference frame. For the remaining frames in the group, we pair up the tokens corresponding to them one by one with the tokens of the reference frame and calculate their similarity. If this similarity exceeds the threshold, then we can use the computation result of the token on the reference frame to replace this token. As shown in step-1 of Fig. 4(a), we select one group of frames as an example. Here we select the 1st frame as the reference frame. The similarity of  $(F_2, T_2)$ ,  $(F_2, T_3)$ , and  $(F_3, T_4)$  exceed the threshold, so they are all ignored in the following computation. For *intra-frame sparsification*, we first divide the tokens in each frame into chunks of  $K$  tokens. For each chunk, we select one token as the reference token. For the remaining tokens in the chunk, we calculate the similarity with the reference token. If this similarity exceeds the threshold, then it indicates that this token has a high similarity in space with the reference token. Therefore, we use the reference token to replace the calculation of this token. If a certain



**Figure 4: The overview of the spatial-temporal activation sparsification method, which includes (a) activation sparsification process and (b) activation recovery process.**

token has been pruned in the inter-frame sparsification, the similarity is 0. As shown in step-2 of Fig. 4(b), we can further reduce the computations of three tokens (i.e.,  $(F_1, T_1)$ ,  $(F_2, T_3)$ , and  $(F_3, T_2)$ ).

**4.1.2 Activation recovery.** For linear operations (e.g., QKVO projection, and FFN), we can use the remaining activation to perform necessary computations, thus reducing the computation. However, for some non-linear operations and  $Q \times K^T$ ,  $S \times V$  operations, we need to restore the entire activation to ensure correctness. In Fig. 4(b), corresponding to activation sparsification, the activation recovery process also includes two steps: intra-frame and inter-frame recovery. Based on the Index Table obtained in the sparsification process, we obtain the corresponding address relationship for activation recovery. We obtain the complete output activation by copying the reference token to the corresponding address.

## 4.2 Operator Analysis

We analyze the operators and computation of the sparse model. From the operator's perspective, similarity calculation requires one inner product and two modulus computations for two input vectors (the dimension is  $d$ ), so its computation can be considered as  $3d$ . Here, we take a single linear operator with generality as an analysis case. Suppose the size of the input activation  $Act_{in}$  is  $F \times T \times d$ , and the size of the weight  $W$  is  $d \times d$ . For the original operator, it is easy to obtain that its computation is  $FTd^2$ . For the sparse operator, we consider the additional computation brought by inter-frame and intra-frame sparsification: (1) Inter-frame sparsification, the number of frames in each group is  $G$ , and there are  $\frac{F}{G}$  groups in total. Therefore, the total computation is  $\frac{F}{G}(G-1)T \cdot 3d$ . (2) Intra-frame sparsification, we divide the input activation of each frame into  $\frac{T}{K}$  chunks with  $K$  tokens. Therefore, the total computation is  $F \frac{T}{K}(K-1) \cdot 3d$ . We assume that the density is  $s$ , so the total computation of the sparse operator is:

$$\begin{aligned} Comp_{compressed} &= s \cdot FTd^2 + \frac{F}{G}(G-1)T \cdot 3d + F \frac{T}{K}(K-1) \cdot 3d \\ &\leq s \cdot FTd^2 + 6FTd \end{aligned}$$

The typical value of the hidden dimension  $d$  in VGMs is over one thousand (e.g., 1152).  $s$  is about 0.3 in our experimental result.

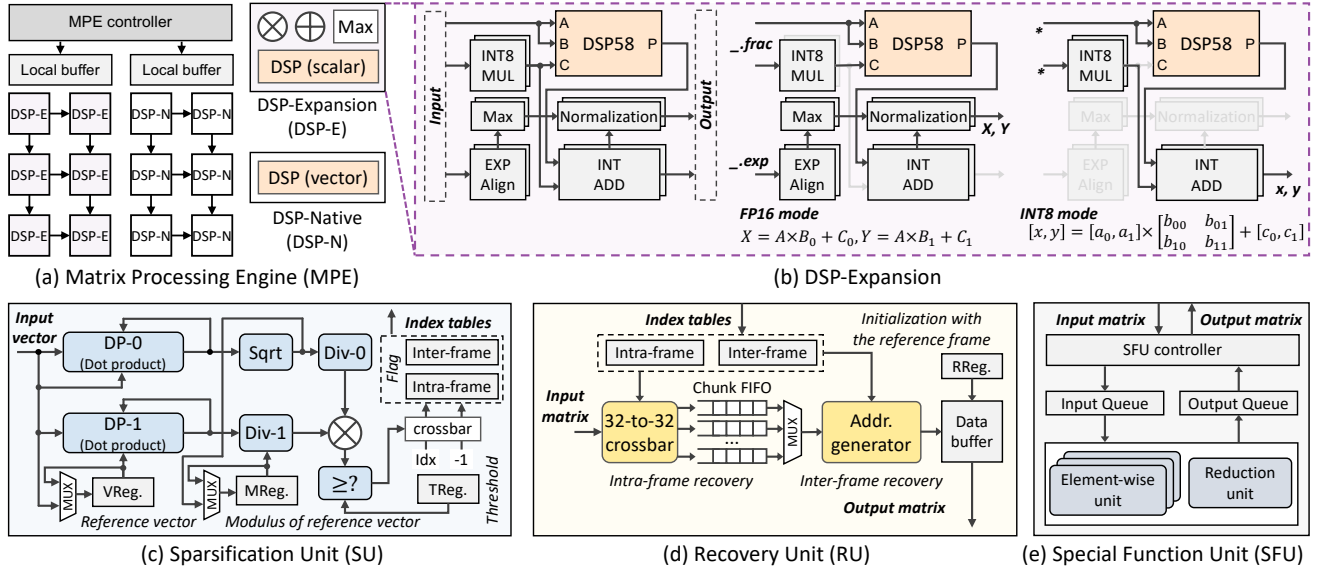
Since the original computation (i.e.,  $FTd^2$ ) has a quadratic term of  $d$ , the additional computation (i.e.,  $6FTd$ ) introduced by sparsification is only a linear term of  $d$ . Therefore, the additional overhead introduced by sparsification can be almost negligible.

## 4.3 Hardware Units

**4.3.1 Matrix Processing Engine.** The matrix processing engine (MPE) efficiently computes linear operators,  $Q \times K^T$ ,  $S \times V$ , and shift&scale operators. As shown in Fig. 5(a), the MPE mainly consists of an MPE controller, a local buffer, and two DSP arrays. The MPE controller drives the DSP arrays to perform corresponding computations according to instructions, and some reusable data will be temporarily stored in the local buffer. There are two main types of DSP arrays: the DSP-E (DSP-Expansion) and DSP-N (DSP-Native) arrays. The DSP-N is based on the native DSP58 and configured as the vector fixed-point ALU mode, which can simultaneously support three INT8 MACs (multiply-accumulate operations). As shown in Fig. 5(b), the DSP-E adds additional circuits based on the DSP58. The DSP-E can be configured in INT8 mode and FP16 mode, which respectively support four INT8 MACs or two FP16 MACs. The DSP-E can reuse the DSP58, INT8 multiplier, and INT adder to maximize hardware efficiency. We describe DSP-E with more detail in Sec. 5.2.

**4.3.2 Sparsification Unit.** The main function of the sparsification unit (SU) is to achieve the online activation sparsification at both inter-frame and intra-frame and generate the Index Tables. As shown in Fig. 5(c), the SU mainly consists of similarity calculation and index generation.

The formula for similarity calculation is as shown in Eq. 1. Since the reference vector  $V_{ref}$  can be reused, we only need to compute its modulus  $\|V_{ref}\|$  once. We can save the computation and memory access overhead by storing  $V_{ref}$  and  $\|V_{ref}\|$  in VReg and MReg, respectively. For  $\|V_{in}\|$ , we first complete its multiplication part on the DP-0 and then complete the square root and reciprocal operations by looking up the table. For  $V_{ref} \cdot V_{in}$ , we first complete its calculation on the DP-1 and then divide it by the pre-compute modulus of the reference vector  $\|V_{ref}\|$  on the Div module. Eventually, the results of these two parts are multiplied together to obtain the similarity between the input vector and the reference vector.



**Figure 5: The hardware architecture of FlightVGM includes (a) Matrix Processing Engine (MPE), (c) Sparsification Unit (SU), (d) Recovery Unit (RU), and (e) Special Function Unit (SFU). MPE mainly consists of the (b) DSP-E array and DSP-N array.**

We compare the similarity with a preset threshold. If the similarity is higher than the threshold, it indicates that the current input vector is highly similar to the reference vector. Therefore, we will write a special mark in the Index Table (2×8KB). The threshold is obtained according to the offline statistical results, which can control the trade-off between the sparsity and the model accuracy.

**4.3.3 Recovery Unit.** The main function of the recovery unit (RU) is to recover the computation results according to the Index Tables to avoid errors in subsequent computations. As shown in Fig. 5(d), the RU mainly consists of two parts: intra-frame recovery and inter-frame recovery. Firstly, RU stores the activation of the reference frame in RReg. Before recovering the activation of each frame, RU initializes the data buffer using the RReg. During the intra-frame recovery stage, RU distributes and multi-casts the input activation according to the information in the intra-frame Index Table

through a crossbar and temporarily stores it in the corresponding chunk FIFO. During the inter-frame recovery stage, the address generator in RU selects the corresponding data from the chunk FIFO according to the information in the inter-frame Index Table and writes it into the data buffer. Finally, when a complete frame has been recovered, the RU outputs the results on the data buffer.

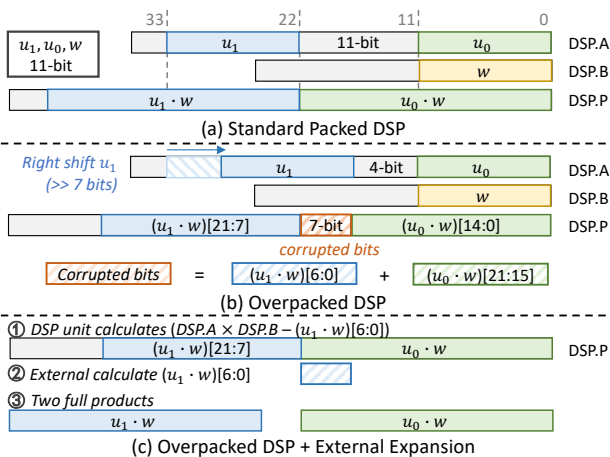
**4.3.4 Special Function Unit.** The Special Function Unit (SFU) processes the non-linear operations on VGMs besides matrix multiplication, including softmax, layer normalization, etc. These operations mainly include element-wise operations and reduction operations. As shown in Fig. 5(e), the SFU mainly consists of multiple element-wise units and one reduction unit, which are executed in a pipelined manner. For example, in the MSA/MCA computation, we achieve parallelism at the vector level for matrix multiplication operations and non-linear operations.

## 5 Hybrid Precision DSP58 Architecture

### 5.1 Overpacking Design for FP16

The AMD V80 FPGA is equipped with the hardware IP DSP58, which can be configured into multiple computing modes [6]. Specifically, DSP58 has configurations such as scalar, vector, and floating-point. However, runtime reconfiguration among these configurations is not achievable, so we can only choose one among them. This problem conflicts with the requirements of VGMs for data precision, resulting in our inability to fully utilize all the computing performance of DSP58.

As shown in Fig. 6(a), we show the calculation of two FP16 multiplication (e.g.,  $A \times B_1$  and  $A \times B_0$ ) as a case, and perform fraction calculations on the DSP58. Since the fraction of an FP16 number is 10-bit, we denote  $u_1 = \{1'b1, B_1.fraction\}$ ,  $u_0 = \{1'b1, B_0.fraction\}$ , and  $w = \{1'b1, A.fraction\}$ . This is equivalent to performing two 11×11-bit multiplications. Simply adopting a standard DSP packing approach like in Xilinx white paper [25] would require at least a



**Figure 6: DSP Overpacking for FP16 fraction multiplication.**

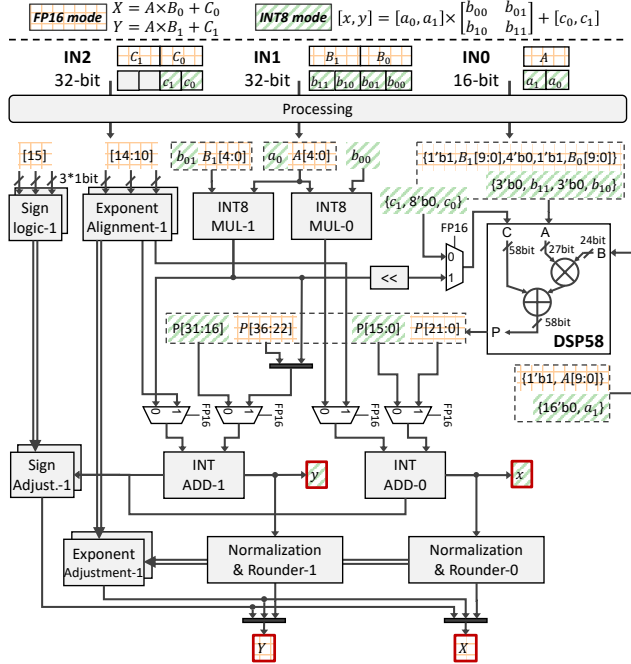


Figure 7: The detailed design of DSP-E.

33×11-bit multiplication to achieve two clean products. However, DSP58 with scalar fixed-point ALU mode only supports 27×24-bit multiplication, so it is impossible to use the method in [25].

We use the overpacking method to recover the corrupted bits of the output DSP58.P port by introducing the additional logic like in [49]. As shown in Fig. 6(b), to fit two 11-bit numbers within 27 bits of the A input with maximum spacing, we overpack the left multiplicand  $u_1$  by 7 bits. Then, the corrupted bits of the result are from the 7 lowest bits of  $u_1 \cdot w$  and the 7 highest bits of  $u_0 \cdot w$ . As shown in Fig. 6(c), to recover these corrupted bits, we add external logic to calculate  $r = (u_1 \cdot w)[6:0]$  then subtract ( $r \ll 16$ ) from the output of the internal multiplier.

## 5.2 DSP58-E Architecture

We propose DSP-Expansion (DSP-E), an innovative FP16-INT8 hybrid precision hardware architecture based on the DSP58. As shown in Fig. 7, DSP-E allows us to configure at runtime and can support two FP16 MACs or four INT8 MACs. Our key insight is to use *additional multipliers to solve the data confusion problem when one DSP58 computes the two multiplications of fraction, and these multipliers can also be reused in INT8 mode*. Specifically, we enable the DSP58 with scalar configuration and some additional configurable circuit designs to provide higher computing performance. In addition to DSP58, DSP-E mainly includes two INT8 multipliers, two INT adders, and some modules for floating-point addition.

In FP16 mode, DSP58 computes the multiplication of the fraction of these two MACs and subtracts the result of the INT8 multiplier to obtain the clean fraction of the intermediate result (i.e.,  $A \times B_0$  and  $A \times B_1$ ). The exponent of the intermediate result is calculated by the INT adder, and then the decimal point position is aligned with the input accumulation term through the exponent alignment and exponent adjust units. Finally, we get the final two results, X

and Y, obtained after the addition operation, normalization, and rounder [19, 28, 36, 67].

In INT8 mode, DSP-E reuses the DSP58, two INT8 multipliers, and two INT adders to maximize the PCP. We input the concatenation of  $b_{11}$ , and  $b_{10}$  to the DSP58.A port, the padding of  $a_1$  to the DSP58.B port, and the concatenation of  $c_1$  and  $c_0$  to the DSP58.C port. The INT8 multipliers and INT adders support the remaining two INT8 MACs (i.e.,  $x = a_0 \times b_{00} + psum_0$  and  $y = a_0 \times b_{01} + psum_1$ , here  $psum_0 = a_1 \times b_{10} + c_0$  and  $psum_1 = a_1 \times b_{11} + c_1$ ).

We propose a heterogeneous DSP58 array design to maximize the throughput of the FPGA under the constraints of computing precision and hardware resources. We have the following candidate designs: DSP-N (INT8 or FP16), logical unit (INT8 or FP16), and DSP-E. We use  $R$  to represent the resource cost under a specific design, including DSP, LUT, REG, RAM, etc. The resource constraint on the FPGA that allows MPE to be used is  $R_{limit}$ . We use  $P$  to represent the INT8/FP16 computing performance for the above candidate designs under a specific ratio  $\alpha$ , which means the computing performance ratio of INT8 to FP16. We use the ECP to measure the overall computing performance, which is equal to INT8 computing performance (or  $\alpha \times$  FP16 computing performance). Compared with the native method, we achieve the computing performance improvement of 1.20× by introducing DSP-E design.

## 6 Dynamic-Static Adaptive Scheduling

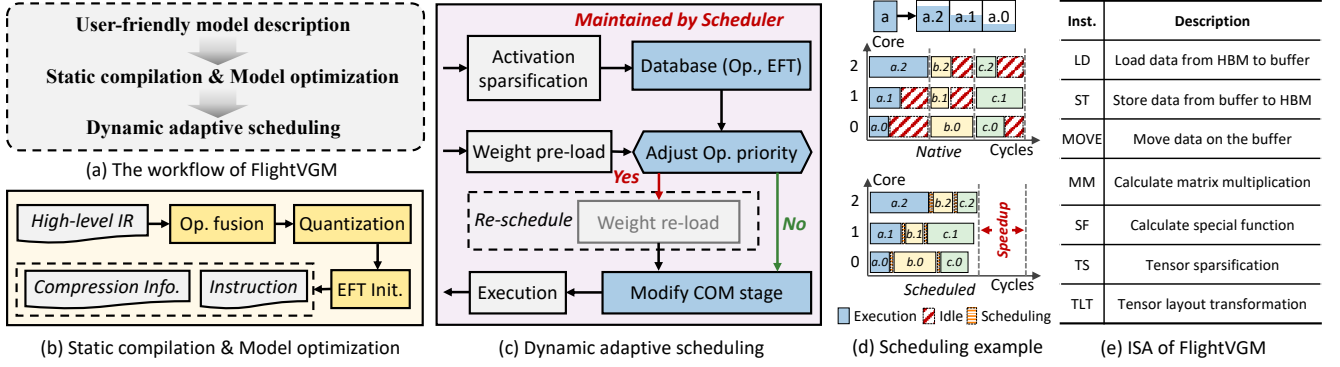
### 6.1 Overview

Fig. 8(a) illustrates the execution flow of FlightVGM. First, we provide a user-friendly programming interface to satisfy the need for model description. Users can easily describe custom models and map weights to specific modules. Subsequently, the compiler converts the model into a high-level intermediate representation (IR), which describes the structural information of the model. The high-level IR is transformed into executable IR during model optimization and static compilation. Model optimization primarily involves two steps: operator fusion and quantization. We perform operation fusion between composable layer pairs and obtain the IR with quantization parameters. In the static compilation, we associate the compression threshold with specific operators and initialize the predicted Earliest Finish Time (EFT).

### 6.2 Instruction Set Architecture

As shown in Fig. 8(e), the ISA of FlightVGM consists of three types: storage management (ST, LD, MOVE), computation (MM, SF), and tensor processing (TS, TLT). The execution of each operator is divided into three stages: weight pre-loading (WL), computation (COM), and tensor processing (TP). The compiler generates three types of instruction sequences to control the aforementioned tasks. The WL instruction sequence is  $LD \rightarrow (MOVE)$ , including two phases: weight loading and on-chip weight movement. Here the instruction with  $()$  is optional. The COM instruction sequence is  $(LD) \rightarrow MM/SF \rightarrow TLT \rightarrow (ST)$ , completing the time-consuming computational tasks and data restoration. The TP instruction sequence is  $(LD) \rightarrow TS \rightarrow TLT \rightarrow (ST)$ , performing activation sparsification and converting the compressed tensor into a dense matrix format usable by the MPE.





**Figure 8: The dynamic-static adaptive scheduling method, which includes (a) overview, (b) Static compilation & Model optimization, (c) Dynamic adaptive scheduling, (d) An example of dynamic scheduling, (e) Instruction Set Architecture.**

### 6.3 Dynamic Adaptive Scheduling

As shown in Fig. 8(c), we propose the dynamic adaptive scheduling method, which simply adjusts the execution order of operators based on the actual workload to enhance computational utilization. Since VGMs are compute-bound, the latency benefits from dynamic-static adaptive scheduling outweigh potential additional memory access overhead, effectively addressing scheduling challenges brought by online sparsification. We consider operators with higher actual EFT to have higher scheduling priority [56]. If the actual EFT is unavailable, sorting is based on the predicted EFT. The scheduler maintains a database composed of operator types, predicted EFT, and actual EFT. It adjusts operator priorities in real time according to activation sparsification results. As shown in Fig. 8(d), dynamic scheduling can significantly improve computational utilization compared with native mapping.

The strategies of the scheduler include: (1) Pre-loading weights as early as possible to overlap the WL and COM stages. (2) Prioritizing the execution of higher-priority operators, assigning them to the earliest available computing cores. Before the COM stage of the current operator (denoted as  $Op_A$ ), the actual EFT of  $Op_A$  may be undetermined. The scheduler executes WL phase of  $Op_A$  based on the predicted EFT (static information). As the TP of input activation is completed, the scheduler obtains the actual EFT of  $Op_A$ . If the priority of  $Op_A$  changes due to a shorter actual EFT, the scheduler enters the re-schedule phase, and executes the WL phase of a newly prioritized operator. Finally, the online scheduler modifies the instruction sequence based on the scheduling results, directing the cores to complete the COM phase of the scheduled operator.

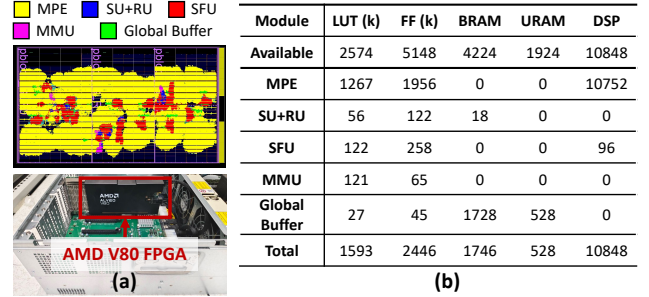
## 7 Evaluation

### 7.1 Evaluation Setup

**7.1.1 Models and Datasets.** We evaluate FlightVGM on two advanced DiT-based VGMs, Latte-1 [43] and Open-Sora 1.2 [72]. Latte-1 is a model trained primarily on human action video sets, and is the first work to explore the use of DiTs in video generation tasks. Open-Sora 1.2 is a widely used open-source model that can generate high-resolution videos of arbitrary aspect ratios. We evaluate the models on UCF-101 [54], a generic dataset of human action poses, and demo prompts from the [14, 15].

**Table 1: Hardware parameters of GPU and FPGA platforms.**

| Platform        | NVIDIA       | AMD       | AMD      |
|-----------------|--------------|-----------|----------|
|                 | 3090 GPU     | U280 FPGA | V80 FPGA |
| Compute units   | 328          | 9024      | 10848    |
|                 | Tensor Cores | DSP48s    | DSP58s   |
| Frequency (MHz) | 1695         | 225       | 300      |
| FP16 PCP (TOPS) | 142          | 5.4       | 6.5      |
| Memory (GB)     | 24           | 8 & 32    | 32 & 32  |
| BW (GB/s)       | 936          | 38 & 460  | 51 & 819 |



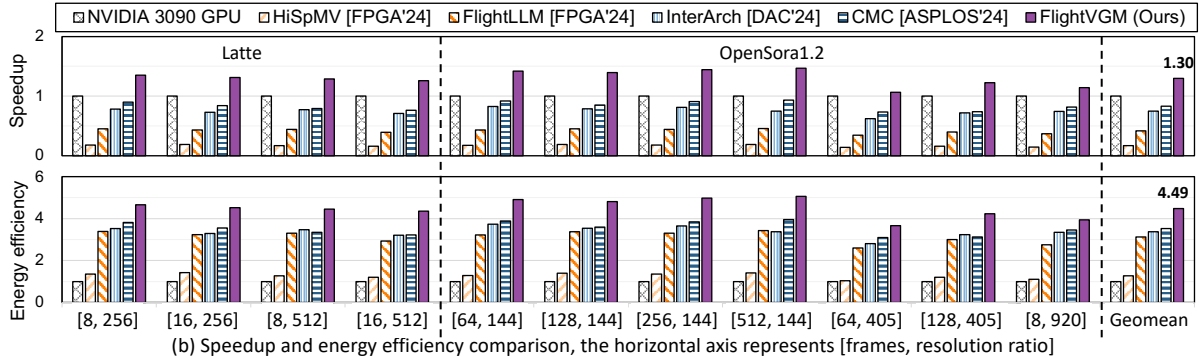
**Figure 9: (a) The implementation layout, board card, and (b) resource usage of FlightVGM on V80 FPGA.**

**7.1.2 Metrics.** We evaluate the accuracy of the VGMs with two representative metrics, CLIPSIM [62] and VBench [37], as used in previous studies [34, 48, 73, 75]. We also use CLIPSIM to measure the prompt-video alignment [3, 4] and VBench to evaluate the overall quality of the generated videos.

**7.1.3 Baselines.** Table 1 shows the parameters of evaluation-related hardware platforms. We select three representative baselines: general-purpose hardware, transformer-based FPGA accelerators, and DiT-based ASIC accelerators. (1) We choose NVIDIA 3090 GPU [2] as the general-purpose hardware baseline. We run native VGMs in FP16 precision to obtain the actual execution time and evaluate the power consumption using NVIDIA's system management interface (nvidia-smi) [1]. (2) For transformer-based FPGA accelerators, We select two SOTA studies, HiSpMV [47] and FlightLLM [66], both implemented on AMD Alveo U280 FPGA [7]. We consider the same activation sparsification and hybrid precision optimization. The hardware parameters and power consumption of HiSpMV and FlightLLM are provided in their papers. (3) We select several

| VGMs          | Compression         | CLIPSIM $\uparrow$ | VBench $\uparrow$ |                     | Coffee cup                                                                        | Coast                                                                               |
|---------------|---------------------|--------------------|-------------------|---------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Open-Sora 1.2 | Original            | 0.302              | 0.639             | Original            |  |  |
|               | INT8 quantization   | 0.213              | 0.602             | INT8 quantization   |  |  |
|               | Hybrid precision    | 0.299              | 0.636             |                     |  |  |
|               | Act. sparsification | 0.296              | 0.629             |                     |  |  |
|               | FlightVGM (Ours)    | 0.295              | 0.627             | Hybrid precision    |  |  |
| Latte         | Original            | 0.197              | 0.572             | Act. sparsification |  |  |
|               | INT8 quantization   | 0.179              | 0.549             |                     |  |  |
|               | Hybrid precision    | 0.195              | 0.572             |                     |  |  |
|               | Act. sparsification | 0.191              | 0.569             | FlightVGM (Ours)    |  |  |
|               | FlightVGM (Ours)    | 0.190              | 0.567             |                     |  |  |

(a) The model accuracy of VGMs and example videos under different compression methods



(b) Speedup and energy efficiency comparison, the horizontal axis represents [frames, resolution ratio]

**Figure 10: (a) The model accuracy of VGMs and example videos under different compression methods. The example videos are generated by the Open-Sora 1.2 model, and the corresponding prompts are from the 4th and 8th lines of [17]. (b) We evaluate the performance and energy efficiency of FlightVGM on different workloads and normalize them to the NVIDIA 3090 GPU.**

SOTA DiT-based ASIC accelerators, including InterArch [59] and CMC [53]. For a fair comparison, we use their sparsification method and align the hardware parameter configurations with those of FlightVGM, including PCP, memory bandwidth, and on-chip buffer capacity. The power consumption of them is also scaled correspondingly. For (2) and (3), since they do not support the latest Latte-1 and Open-Sora 1.2 model, we build a cycle-accurate performance simulator to evaluate their performance. The gap between the results of the simulator and their original data is less than 6%.

**7.1.4 FPGA Implementation.** We implement FlightVGM on the AMD V80 FPGA [9]. Each SLR corresponds to a computing core, and they perform computing tasks under the control of the embedded CPU. The V80 FPGA has a 32GB HBM with 819GB/s and a 32GB DDR memory with 32GB/s. FlightVGM achieves equivalent INT8 PCP of 20.3TOPS and FP16 PCP of 2.8TOPS with the 300MHz frequency. Fig. 9 is the implementation layout, board card, and resource usage of FlightVGM on the V80 FPGA, respectively. The latency of the FPGA comes from the actual running results. We use the `ami_tool` [10] to measure the power consumption.

## 7.2 Evaluation of Model Accuracy

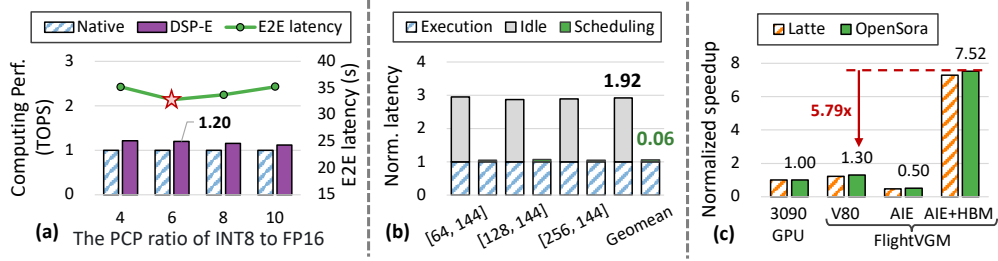
We evaluate the model accuracy and example video with different compression methods in Fig. 10(a). We take the original dense model with FP16 precision as the baseline, and FlightVGM includes both hybrid precision quantization and activation sparsification. For the hybrid precision method, we quantize all linear layers in the DiT

blocks and maintain the attention map computation and modules outside the DiT (e.g., embedding layers) at FP16 precision. We also conduct a comparison with INT8 quantization for all layers. For the activation sparsification, we adopt the threshold of 0.95/0.98 for QKV/O projection and 0.92 for FFN. With the efficient hardware architecture, we achieve a 3.17 $\times$  end-to-end acceleration. Compared with the baseline, the model accuracy of FlightVGM is almost the same (only average 0.008 loss), while INT8 quantization has an average 0.042 loss. As shown in the upper right of Fig. 10(a), we test the actual video generation for the above different compression methods. We find that the video quality generated by FlightVGM is almost the same as that of the original model.

## 7.3 Evaluation of Performance

We compare the speedup and energy efficiency of FlightVGM to NVIDIA 3090 GPU and the current SOTA accelerators on the Latte-1 and Open-Sora 1.2 models in Fig. 10. Overall, FlightVGM achieves an average 2.84 $\times$  performance and 1.71 $\times$  energy efficiency improvements compared with the SOTA accelerators.

**7.3.1 Performance with GPUs.** For the NVIDIA 3090 GPU, the V80 FPGA has a significant computing performance gap of over 21 $\times$  under the FP16 precision. Nevertheless, the FlightVGM implemented based on the V80 FPGA still surpasses the GPUs, including performance and energy efficiency. This is because FPGAs can fully exploit the inherent sparsity and hybrid precision of VGM to achieve acceleration, but GPUs are difficult to achieve acceleration due to the limitations of their hardware architectures. Compared with NVIDIA



**Figure 11: Ablation studies for (a) DSP-E architecture and (b) adaptive scheduling. (c) Discussions for AIE-based implementation.**

3090 GPU, FlightVGM achieves 1.30 $\times$  and 4.49 $\times$  improvement in performance and energy efficiency respectively.

**7.3.2 Performance with SOTA Accelerators.** For the DiT-based accelerators, we align the hardware parameters with FlightVGM regarding PCP and memory bandwidth for a fair comparison. As shown in Fig. 10(b), compared with InterArch/CMC, FlightVGM achieves 1.74/1.56 $\times$  performance and 1.33/1.27 $\times$  energy efficiency improvements, respectively. For InterArch and CMC, FlightVGM achieves a further reduction in computational cost through more efficient compression algorithms and hybrid precision quantization, thus having better performance.

For the transformer-based FPGA Accelerators, we consider the sparse support reported in their papers [47, 66]. Compared with HiSpMV, FlightVGM achieves 7.69 $\times$  speedup and 3.52 $\times$  energy efficiency improvement. This is because HiSpMV cannot efficiently handle the large number of matrix multiplications in VGM due to its low parallelism. Compared with FlightLLM, FlightVGM achieves 3.12 $\times$  speedup and 1.43 $\times$  energy efficiency improvement. This is because FlightLLM cannot effectively support online sparsification and lacks optimization in computing performance.

## 7.4 Ablation Studies and Discussion

**7.4.1 Effect of DSP-E Architecture.** We compare the hardware computing performance with or without the DSP-E design under the same hardware resource constraints. As shown in Fig. 11(a), compared with native design (*i.e.*, not using DSP-E), DSP-E effectively increases the computing performance by 1.20 $\times$ . We also find that different computing performance ratios significantly impact end-to-end latency. Specifically, if the ratio is lower/higher, then the end-to-end latency is easily bottlenecked by INT8/FP16 computing performance. Therefore, when we choose a ratio of 6, it is the optimal point, which is determined by the structure of VGM. Considering the advantages brought by hybrid precision, we can increase the overall equivalent computing performance by 3.26 $\times$  compared to simply using floating-point computing performance.

**7.4.2 Effect of Adaptive Scheduling.** As shown in Fig. 11(b), we compare the latency and its breakdown before and after introducing adaptive scheduling and normalize it according to the valid execution time. Due to the workload imbalance after online sparsification, the average idle rate of PEs is 65.8%. We eliminate waste through adaptive scheduling, and the additional overhead introduced is negligible. Adaptive scheduling achieves the average 2.75 $\times$  PE utilization improvement.

**7.4.3 Discussion of AIE-based FPGA.** There are also FPGAs equipped with AIE (AI Engine) to achieve higher computing performance, for example, AMD VEK280 FPGA [8] and VCK190 FPGA used in SSR [74]. We also evaluate the FlightVGM (AIE-based) compared with NVIDIA 3090 GPU [5] in Fig. 11(c). The VEK280 FPGA provides 228 TOPS INT8 computing performance but only has 86.7 GB/s memory bandwidth. Compared with the FlightVGM (V80), FlightVGM (AIE-based) cannot effectively accelerate the VGM due to the low memory bandwidth. Further, FlightVGM (AIE+HBM) can further increase the acceleration by 5.79 $\times$  on the Open-Sora 1.2 model if the existing AIE-based FPGA is equipped with HBM.

## 8 Related Work

For the LLM accelerations [33, 39, 58], FlightLLM [66] proposes a configurable sparse DSP chain to accelerate sparse LLMs, enabling running LLaMA2-7B on a single AMD U280 FPGA. Chen *et al.* [21] provide an HLS-based library for mapping LLMs onto cloud FPGAs. Li *et al.* [39] further summarizes existing LLM inference acceleration between various hardware platforms. For the vision transformer, HeatViT [23] applies adaptive token pruning and INT8 quantization to map ViT onto FPGAs. SSR [74] proposes a spatial-sequential-hybrid architecture to accelerate ViT on both AIEs and FPGAs, achieving a throughput of 8.21-10.90TOPS on the AMD VCK190 board. However, none of the above studies can provide sufficient throughput to meet the INT8/FP16 computing requirement of VGMs using existing FPGA platforms. Besides, Recent studies have made great progress in accelerating sparse matrix multiplication on HBM-based cloud FPGAs [24, 26, 38, 47, 51, 52]. However, these studies do not consider the coarse-grained spatial-temporal redundancy in VGMs, resulting in additional logic overhead and limited PCP.

## 9 Conclusion

In this paper, we propose **FlightVGM**, the first FPGA accelerator for efficient VGM inference with online sparsification and hybrid precision. Specifically, we achieve efficient VGM inference by computation saving, increasing the PCP of FPGA, and improving computational utilization. FlightVGM surpasses NVIDIA 3090 GPU by 1.30 $\times$  in performance and 4.49 $\times$  in energy efficiency, respectively.

## Acknowledgments

This work was sponsored by the National Natural Science Foundation of China (No. 62325405, 62104128, U21B2031), Shanghai Rising-Star Program (No. 24QB2706200), Tsinghua EE Xilinx AI Research Fund, Beijing National Research Center for Information Science and Technology (No. BNR2024TD03001). We thank the insightful discussion with colleagues at Infinigence-AI.

## References

- [1] 2013. NVIDIA-SMI. [https://docs.nvidia.com/deploy/pdf/NVSMI\\_Manual.pdf](https://docs.nvidia.com/deploy/pdf/NVSMI_Manual.pdf).
- [2] 2020. NVIDIA AMPERE GA102 GPU ARCHITECTURE. Retrieved September 25, 2024 from <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>
- [3] 2021. openai/clip-vit-base-patch32. Retrieved September 7, 2024 from <https://huggingface.co/openai/clip-vit-base-patch32>
- [4] 2021. sentence-transformers/clip-ViT-B-32. Retrieved September 7, 2024 from <https://huggingface.co/sentence-transformers/clip-ViT-B-32>
- [5] 2022. NVIDIA ADA GPU ARCHITECTURE. Retrieved September 7, 2024 from <https://images.nvidia.com/aem-dam/Solutions/Data-Center/14/nvidia-ada-gpu-architecture-whitepaper-v2.1.pdf>
- [6] 2022. Versal ACAP DSP Engine Architecture Manual (AM004). Retrieved September 7, 2024 from <https://docs.amd.com/r/en-US/am004-versal-dsp-engine>
- [7] 2023. Alveo U280 Data Center Accelerator Card Data Sheet (DS963). Retrieved September 25, 2024 from <https://docs.amd.com/r/en-US/ds963-u280/Summary>
- [8] 2023. Versal AI Edge Series VEK280 Evaluation Kit. Retrieved September 27, 2024 from <https://www.xilinx.com/products/boards-and-kits/vek280.html>
- [9] 2024. Alveo V80 Compute Accelerator Card. Retrieved September 7, 2024 from <https://www.amd.com/content/dam/amd/en/documents/products/accelerators/alveo/v80/alveo-v80-product-brief.pdf>
- [10] 2024. AVED Management Interface userguide (ami\_tool). Retrieved September 27, 2024 from [https://xilinx.github.io/AVED/amd\\_v80\\_gen5x8\\_exdes\\_2\\_20240408/AVED%2BManagement%2BInterface%2BUserguide%2B%28ami\\_tool%29.html](https://xilinx.github.io/AVED/amd_v80_gen5x8_exdes_2_20240408/AVED%2BManagement%2BInterface%2BUserguide%2B%28ami_tool%29.html)
- [11] 2024. chatgpt. Retrieved September 7, 2024 from <https://openai.com/chatgpt/>
- [12] 2024. Dreamina: Free AI Image Generator. Retrieved September 7, 2024 from <https://dreamina.capcut.com/>
- [13] 2024. Kling. Retrieved September 7, 2024 from <https://kling.kuaishou.com/>
- [14] 2024. Latte gallery. Retrieved June 17, 2024 from [https://maxin-cn.github.io/latte\\_project/](https://maxin-cn.github.io/latte_project/)
- [15] 2024. Open-Sora gallery. Retrieved June 17, 2024 from <https://hpaitech.github.io/Open-Sora/>
- [16] 2024. Sora. Retrieved September 7, 2024 from <https://openai.com/index/sora/>
- [17] 2024. t2v\_sora.txt. Retrieved September 27, 2024 from [https://github.com/hpaitech/Open-Sora/blob/main/assets/texts/t2v\\_sora.txt](https://github.com/hpaitech/Open-Sora/blob/main/assets/texts/t2v_sora.txt)
- [18] 2024. Veo. Retrieved September 7, 2024 from <https://deepmind.google/technologies/veo/>
- [19] Nicolas Brunie. 2017. Modified fused multiply and add for exact low precision product accumulation. In 2017 IEEE 24th Symposium on Computer Arithmetic (ARITH). IEEE, 106–113.
- [20] Yihan Cao, Siyu Li, Yixin Liu, Zhiling Yan, Yutong Dai, Philip S Yu, and Lichao Sun. 2023. A comprehensive survey of ai-generated content (aigc): A history of generative ai from gan to chatgpt. arXiv preprint arXiv:2303.04226 (2023).
- [21] Hongzheng Chen, Jiahao Zhang, Yixiao Du, Shaojie Xiang, Zichao Yue, Niansong Zhang, Yaohui Cai, and Zhiru Zhang. 2024. Understanding the potential of fpga-based spatial acceleration for large language model inference. ACM Transactions on Reconfigurable Technology and Systems (2024).
- [22] Peng Chen, Weichen Liu, Xu Jiang, Qingqiang He, and Nan Guan. 2019. Timing-Anomaly Free Dynamic Scheduling of Conditional DAG Tasks on Multi-Core Systems. ACM Trans. Embed. Comput. Syst. 18, 5s, Article 91 (Oct. 2019), 19 pages. <https://doi.org/10.1145/3358236>
- [23] Peiyan Dong, Mengshu Sun, Alec Lu, Yanyue Xie, Kenneth Liu, Zhenglun Kong, Xin Meng, Zhengang Li, Xue Lin, Zhenman Fang, et al. 2023. Heatvit: Hardware-efficient adaptive token pruning for vision transformers. In 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE, 442–455.
- [24] Yixiao Du, Yuwei Hu, Zhongchun Zhou, and Zhiru Zhang. 2022. High-performance sparse linear algebra on hbm-equipped fpgas using hls: A case study on spmv. In Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 54–64.
- [25] Yao Fu, Ephrem Wu, Ashish Sirasao, Sedny Attia, Kamran Khan, and Ralph Wittig. 2017. Deep Learning with INT8 Optimization on Xilinx Devices. (2017).
- [26] Yingxue Gao, Lei Gong, Chao Wang, Teng Wang, Xi Li, and Xuehai Zhou. 2023. Algorithm/hardware co-optimization for sparsity-aware SpMM acceleration of GNNs. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 42, 12 (2023), 4763–4776.
- [27] Ben Goertzel. 2014. Artificial general intelligence: concept, state of the art, and future prospects. Journal of Artificial General Intelligence 5, 1 (2014), 1.
- [28] Azzam Haidar, Stanimire Tomov, Jack Dongarra, and Nicholas J Higham. 2018. Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers. In SC18: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 603–613.
- [29] Song Han, Junlong Kang, Huizi Mao, Yiming Hu, Xin Li, Yubin Li, Dongliang Xie, Hong Luo, Song Yao, Yu Wang, et al. 2017. ESE: Efficient speech recognition engine with sparse lstm on fpga. In Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 75–84.
- [30] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A Horowitz, and William J Dally. 2016. EIE: Efficient inference engine on compressed deep neural network. ACM SIGARCH Computer Architecture News 44, 3 (2016), 243–254.
- [31] Qingqiang He, Xu Jiang, Nan Guan, and Zhishan Guo. 2019. Intra-Task Priority Assignment in Real-Time Scheduling of DAG Tasks on Multi-Cores. IEEE Transactions on Parallel and Distributed Systems 30, 10 (2019), 2283–2295. <https://doi.org/10.1109/TPDS.2019.2910525>
- [32] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. Advances in neural information processing systems 33 (2020), 6840–6851.
- [33] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Yuhang Dong, Yu Wang, et al. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. Proceedings of Machine Learning and Systems 6 (2024), 148–161.
- [34] Wenyi Hong, Ming Ding, Wendi Zheng, Xinghan Liu, and Jie Tang. 2022. Cogvideo: Large-scale pretraining for text-to-video generation via transformers. arXiv preprint arXiv:2205.15868 (2022).
- [35] Guyue Huang, Zhengyang Wang, Po-An Tsai, Chen Zhang, Yufei Ding, and Yuan Xie. 2023. RM-STC: Row-Merge Dataflow Inspired GPU Sparse Tensor Core for Energy-Efficient Sparse Acceleration. In Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. 338–352.
- [36] Libo Huang, Sheng Ma, Li Shen, Zhiyong Wang, and Nong Xiao. 2011. Low-cost binary 128 floating-point FMA unit design with SIMD support. IEEE Trans. Comput. 61, 5 (2011), 745–751.
- [37] Ziqi Huang, Yinan He, Jiahuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, et al. 2024. Vbench: Comprehensive benchmark suite for video generative models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 21807–21818.
- [38] Abhishek Kumar Jain, Chirag Ravishankar, Hossein Omidian, Sharan Kumar, Maithilee Kulkarni, Aashish Tripathi, and Dinesh Gaitonde. 2023. Modular and lean architecture with elasticity for sparse matrix vector multiplication on fpgas. In 2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). IEEE, 133–143.
- [39] Jinhao Li, Jiaming Xu, Shan Huang, Yonghua Chen, Wen Li, Jun Liu, Yaoyuan Lian, Jiayi Pan, Li Ding, Hao Zhou, et al. 2024. Large language model inference acceleration: A comprehensive hardware perspective. arXiv preprint arXiv:2410.04466 (2024).
- [40] Wenxuan Liu and Sai Qian Zhang. 2024. HQ-DiT: Efficient Diffusion Transformer with FP4 Hybrid Quantization. arXiv:2405.19751 [cs.CV] <https://arxiv.org/abs/2405.19751>
- [41] Yajing Liu, Ruiqi Chen, Shuyang Li, Jing Yang, Shun Li, and Bruno da Silva. 2024. FPGA-Based Sparse Matrix Multiplication Accelerators: From State-of-the-art to Future Opportunities. ACM Transactions on Reconfigurable Technology and Systems (2024).
- [42] Yaofang Liu, Xiaodong Cun, Xuebo Liu, Xintao Wang, Yong Zhang, Haoxin Chen, Yang Liu, Tieyong Zeng, Raymond Chan, and Ying Shan. 2024. Evalcrafter: Benchmarking and evaluating large video generation models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 22139–22149.
- [43] Xin Ma, Yaohui Wang, Gengyun Jia, Xinyuan Chen, Ziwei Liu, Yuan-Fang Li, Cunjun Chen, and Yu Qiao. 2024. Latte: Latent Diffusion Transformer for Video Generation. arXiv preprint arXiv:2401.03048 (2024).
- [44] Alessandra Melani, Marko Bertogna, Vincenzo Bonifaci, Alberto Marchetti-Spaccamela, and Giorgio C. Buttazzo. 2015. Response-Time Analysis of Conditional DAG Tasks in Multiprocessor Systems. In 2015 27th Euromicro Conference on Real-Time Systems. 211–221. <https://doi.org/10.1109/ECRTS.2015.26>
- [45] Alessandra Melani, Marko Bertogna, Vincenzo Bonifaci, Alberto Marchetti-Spaccamela, and Giorgio C. Buttazzo. 2015. Response-Time Analysis of Conditional DAG Tasks in Multiprocessor Systems. In 2015 27th Euromicro Conference on Real-Time Systems. 211–221. <https://doi.org/10.1109/ECRTS.2015.26>
- [46] William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision. 4195–4205.
- [47] Manoj B Rajashekar, Xingyu Tian, and Zhenman Fang. 2024. HiSpMV: Hybrid Row Distribution and Vector Buffering for Imbalanced SpMV Acceleration on FPGAs. In Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays. 154–164.
- [48] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, et al. 2022. Make-a-video: Text-to-video generation without text-video data. arXiv preprint arXiv:2209.14792 (2022).
- [49] Jan Sommer, M. Akif Özkan, Oliver Keszocze, and Jürgen Teich. 2022. DSP-Packing: Squeezing Low-precision Arithmetic into FPGA DSP Blocks. In 2022 32nd International Conference on Field-Programmable Logic and Applications (FPL). 160–166. <https://doi.org/10.1109/FPL57034.2022.00035>



- [50] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2020. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502* (2020).
- [51] Linghao Song, Yuze Chi, Licheng Guo, and Jason Cong. 2022. Serpens: A high bandwidth memory based accelerator for general-purpose sparse matrix-vector multiplication. In *Proceedings of the 59th ACM/IEEE design automation conference*. 211–216.
- [52] Linghao Song, Yuze Chi, Atefeh Sohrabizadeh, Young-kyu Choi, Jason Lau, and Jason Cong. 2022. Sextans: A streaming accelerator for general-purpose sparse-matrix dense-matrix multiplication. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 65–77.
- [53] Zhuoran Song, Chunyu Qi, Fangxin Liu, Naifeng Jing, and Xiaoyao Liang. 2024. CMC: Video Transformer Acceleration via CODEC Assisted Matrix Condensing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 201–215.
- [54] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. 2012. UCF101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402* (2012).
- [55] Gary J Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. 2012. Overview of the high efficiency video coding (HEVC) standard. *IEEE Transactions on circuits and systems for video technology* 22, 12 (2012), 1649–1668.
- [56] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE transactions on parallel and distributed systems* 13, 3 (2002), 260–274.
- [57] Petros Voudouris, Per Stenström, and Risat Pathan. 2017. Timing-Anomaly Free Dynamic Scheduling of Task-Based Parallel Applications. In *2017 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 365–376. <https://doi.org/10.1109/RTAS.2017.2>
- [58] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 97–110.
- [59] Xuhang Wang, Zhuoran Song, and Xiaoyao Liang. 2024. InterArch: Video Transformer Acceleration via Inter-Feature Deduplication with Cube-based Dataflow. In *2024 61st ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [60] Yang Wang, Chen Zhang, Zhiqiang Xie, Cong Guo, Yunxin Liu, and Jingwen Leng. 2021. Dual-side sparse tensor core. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1083–1095.
- [61] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.
- [62] Chenfei Wu, Lun Huang, Qianxi Zhang, Binyang Li, Lei Ji, Fan Yang, Guillermo Sapiro, and Nan Duan. 2021. Godiva: Generating open-domain videos from natural descriptions. *arXiv preprint arXiv:2104.14806* (2021).
- [63] Yannan Nellie Wu, Po-An Tsai, Saurav Muralidharan, Angshuman Parashar, Vivienne Sze, and Joel S Emer. 2023. HighLight: Efficient and Flexible DNN Acceleration with Hierarchical Structured Sparsity. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*.
- [64] Jiaqi Xu, Xinyi Zou, Kunzhe Huang, Yunkuo Chen, Bo Liu, MengLi Cheng, Xing Shi, and Jun Huang. 2024. EasyAnimate: A High-Performance Long Video Generation Method based on Transformer Architecture. *arXiv preprint arXiv:2405.18991* (2024).
- [65] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. 2024. Cogvideox: Text-to-video diffusion models with an expert transformer. *arXiv preprint arXiv:2408.06072* (2024).
- [66] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, et al. 2024. Flightllm: Efficient large language model inference with a complete mapping flow on fpgas. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 223–234.
- [67] Hao Zhang, Hyuk Jae Lee, and Seok-Bum Ko. 2018. Efficient fixed/floating-point merged mixed-precision multiply-accumulate unit for deep learning processors. In *2018 IEEE international symposium on circuits and systems (ISCAS)*. IEEE, 1–5.
- [68] Shuai Zhao, Xiaotian Dai, and Iain Bate. 2022. DAG scheduling and analysis on multi-core systems by modelling parallelism and dependency. *IEEE transactions on parallel and distributed systems* 33, 12 (2022), 4019–4038.
- [69] Shuai Zhao, Xiaotian Dai, Iain Bate, Alan Burns, and Wanli Chang. 2020. DAG Scheduling and Analysis on Multiprocessor Systems: Exploitation of Parallelism and Dependency. In *2020 IEEE Real-Time Systems Symposium (RTSS)*. 128–140. <https://doi.org/10.1109/RTSS49844.2020.00022>
- [70] Tianchen Zhao, Tongcheng Fang, Enshu Liu, Rui Wan, Widyadewi Soedarmadji, Shiyao Li, Zinan Lin, Guohao Dai, Shengen Yan, Huazhong Yang, Xuefei Ning, and Yu Wang. 2024. ViDiT-Q: Efficient and Accurate Quantization of Diffusion Transformers for Image and Video Generation. *arXiv:2406.02540 [cs.CV]* <https://arxiv.org/abs/2406.02540>
- [71] Tianchen Zhao, Xuefei Ning, Tongcheng Fang, Enshu Liu, Guyue Huang, Zinan Lin, Shengen Yan, Guohao Dai, and Yu Wang. 2025. Mixdq: Memory-efficient few-step text-to-image diffusion models with metric-decoupled mixed precision quantization. In *European Conference on Computer Vision*. Springer, 285–302.
- [72] Zangwei Zheng, Xiangyu Peng, Tianji Yang, Chenhui Shen, Shenggui Li, Hongxin Liu, Yukun Zhou, Tianyi Li, and Yang You. 2024. Open-Sora: Democratizing Efficient Video Production for All. <https://github.com/hpcaitech/Open-Sora>
- [73] Daquan Zhou, Weimin Wang, Hanshu Yan, Weiwei Lv, Yizhe Zhu, and Jiashi Feng. 2022. Magicvideo: Efficient video generation with latent diffusion models. *arXiv preprint arXiv:2211.11018* (2022).
- [74] Jinming Zhuang, Zhuoping Yang, Shixin Ji, Heng Huang, Alex K Jones, Jingtong Hu, Yiyu Shi, and Peipei Zhou. 2024. SSR: Spatial Sequential Hybrid Architecture for Latency Throughput Tradeoff in Transformer Acceleration. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 55–66.
- [75] Shaobin Zhuang, Kunchang Li, Xinyuan Chen, Yaohui Wang, Ziwei Liu, Yu Qiao, and Yali Wang. 2024. Vlogger: Make your dream a vlog. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8806–8817.